Support Vector Machine Classification & Manual Feature Engineering

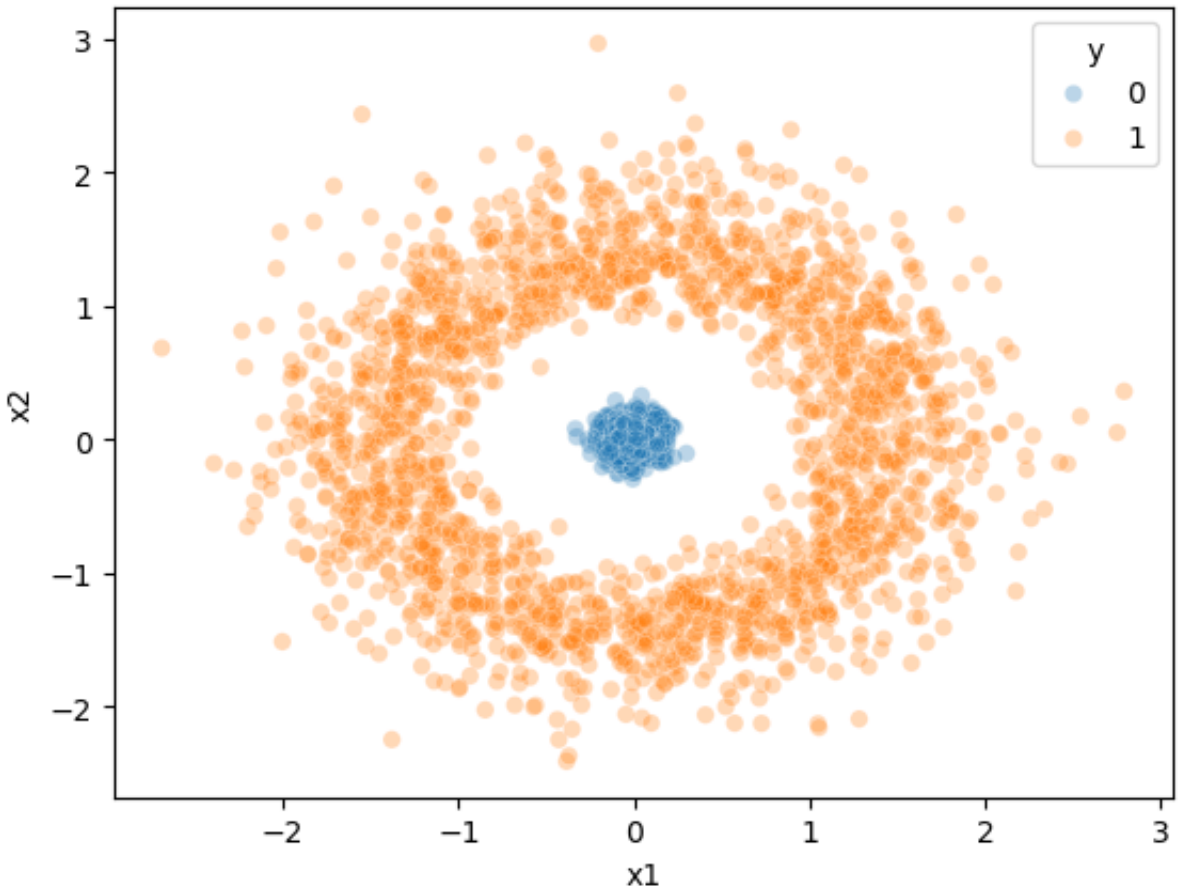
Cameron Batts — SVM · Gamma Tuning · Feature Engineering · Logistic Regression Validation

In [6]:
```
import math
import random

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score
)
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

from utils_DA import plot_decision_boundary
```

In [7]:
```
random.seed(42)
```

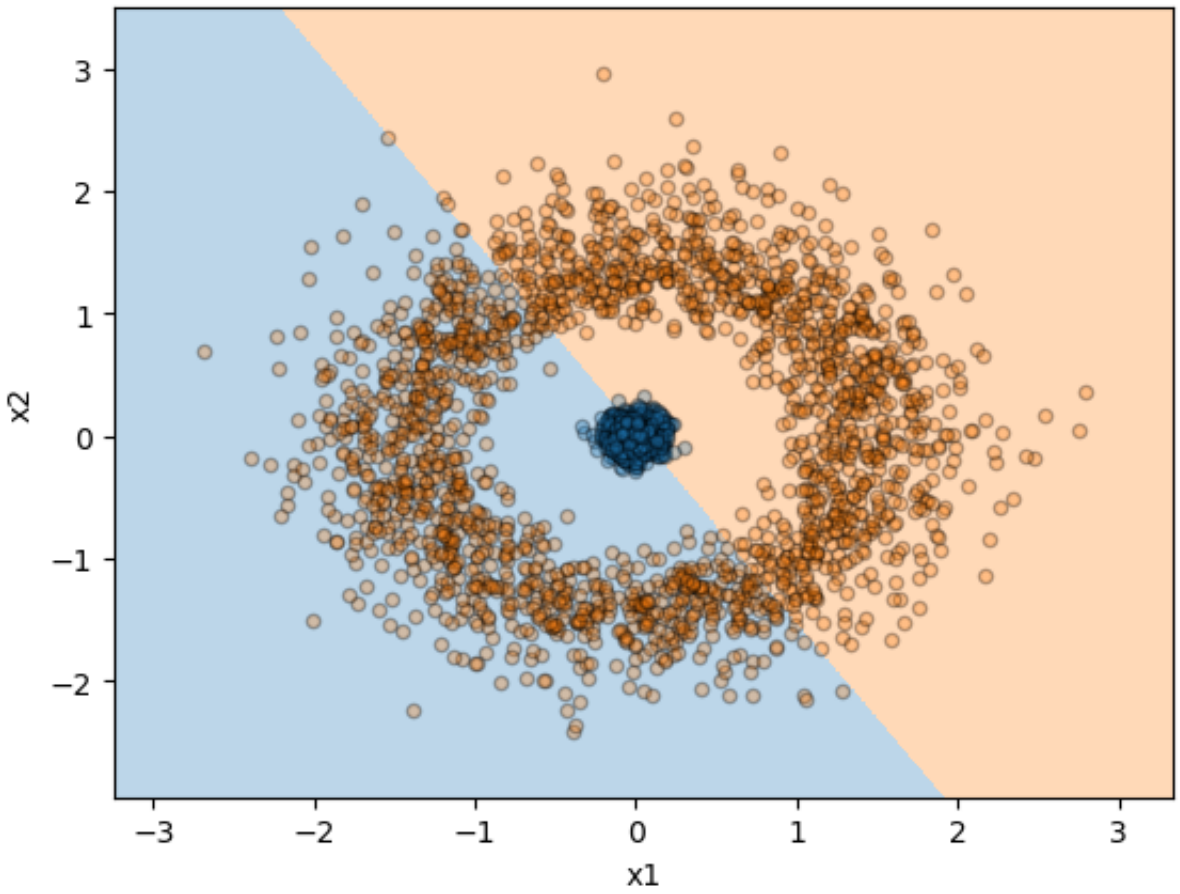
Here, we generate a synthetic dataset for classification with two predictors, `x1` and `x2`.

In [8]:
```
def radial2cartesian(rho, theta):
    return rho * math.cos(theta), rho * math.sin(theta)

n_class0 = 2000
n_class1 = 2000

rho = [random.normalvariate(0.0, 0.10) for _ in range(n_class0)] + \
      [random.lognormvariate(0.4, 0.2) for _ in range(n_class1)]
theta = [random.uniform(0, 2 * math.pi) for _ in range(n_class0 + n_cl

data = pd.DataFrame([radial2cartesian(r, t) for r, t in zip(rho, theta
data['y'] = pd.Series([0] * n_class0 + [1] * n_class1)

_ = sns.scatterplot(x='x1', y='x2', hue='y', data=data, alpha=0.3)
```



In [9]:
```
logistic_regression = LogisticRegression().fit(data[['x1', 'x2']].valu
```

In [10]:
```
plot_decision_boundary(
    logistic_regression,
    data[['x1', 'x2']].values, data['y'].values,
    n_points=500, alpha=0.3, x_label='x1', y_label='x2'
)
```



In [11]:
```
accuracy_score(data['y'], logistic_regression.predict(data[['x1', 'x2'
```

Out[11]: 0.58925

In [12]:
```
precision_score(data['y'], logistic_regression.predict(data[['x1', 'x2
```

Out[12]: 0.6079854809437386

In [13]:
```
recall_score(data['y'], logistic_regression.predict(data[['x1', 'x2']]
```

Out[13]: 0.5025

As expected, the logistic classifier does not perform too well since the two classes are not linearly separable in the predictors space.

Let's see if a Support Vector Machine classifier can learn a transformation of the current predictors such that in the transformed (and potentially higher-dimensional) predictors space the two classes are linearly separable.

In [14]:
```
gamma_candidates = np.logspace(-2, 1, 20)
```

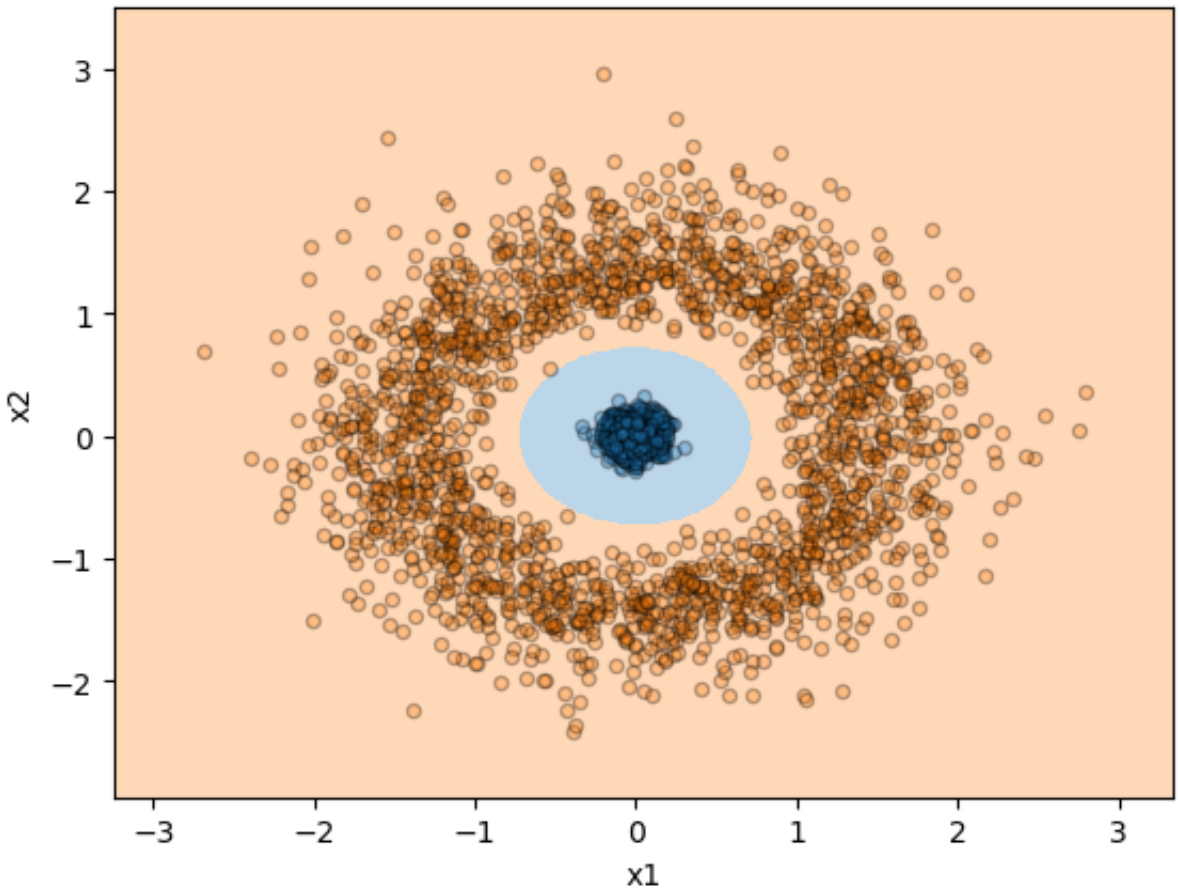
In [15]:
```
param_grid = {
    'svc__gamma': gamma_candidates,
```

```
    'svc__kernel': ['rbf'],
    'svc__C': [1]
}

pipeline = make_pipeline(StandardScaler(), SVC())

grid_search = GridSearchCV(
    pipeline,
    param_grid,
    cv=5
)

grid_search.fit(data[['x1', 'x2']].values, data['y'].values)

svc = grid_search.best_estimator_
```

In [16]:
```
svc = grid_search.best_estimator_
```

Here we plot the decision boundary learned by the SVM.

In [17]:
```
try:
    plot_decision_boundary(
        svc,
        data[['x1', 'x2']].values, data['y'].values,
        n_points=500, alpha=0.3, x_label='x1', y_label='x2'
    )
except NameError:
    print('The object `svc` does not exist!')
```



In [18]:
```
y_pred = svc.predict(data[['x1', 'x2']].values)

accuracy = accuracy_score(data['y'], y_pred)
precision = precision_score(data['y'], y_pred)
recall = recall_score(data['y'], y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
```

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
```

The SVM learned that the classification depends on the distance of each point from the center (0,0).

While a linear model can only draw a straight line, the SVM learned a circular boundary. It identified that points within a certain radius belong to Class 0, while points further out belong to Class 1.

In [19]:
```
data['x3'] = np.sqrt(data['x1']**2 + data['x2']**2)
```

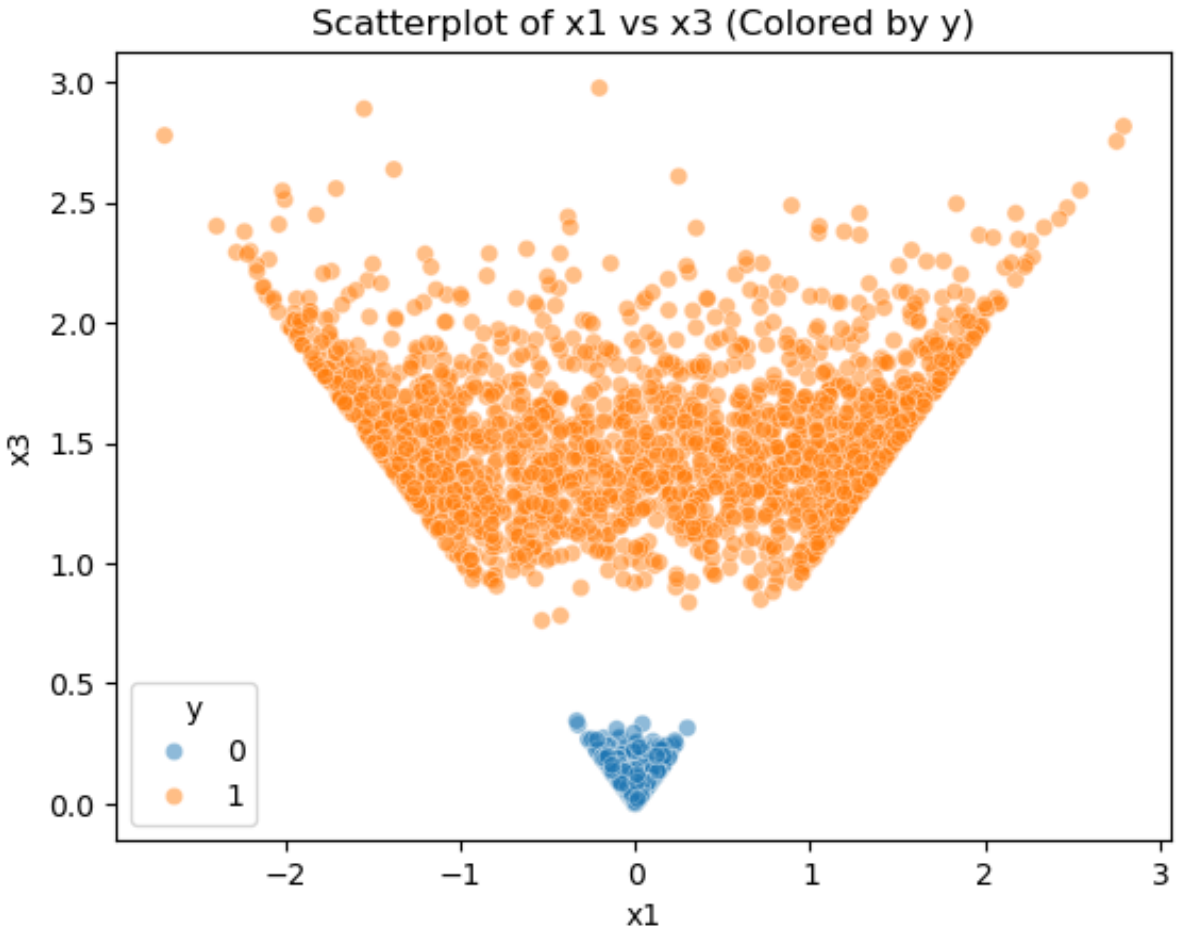
We'll now fit a logistic regression model (which is a linear classifier) on the new set of predictors `x1`, `x2`, `x3`.

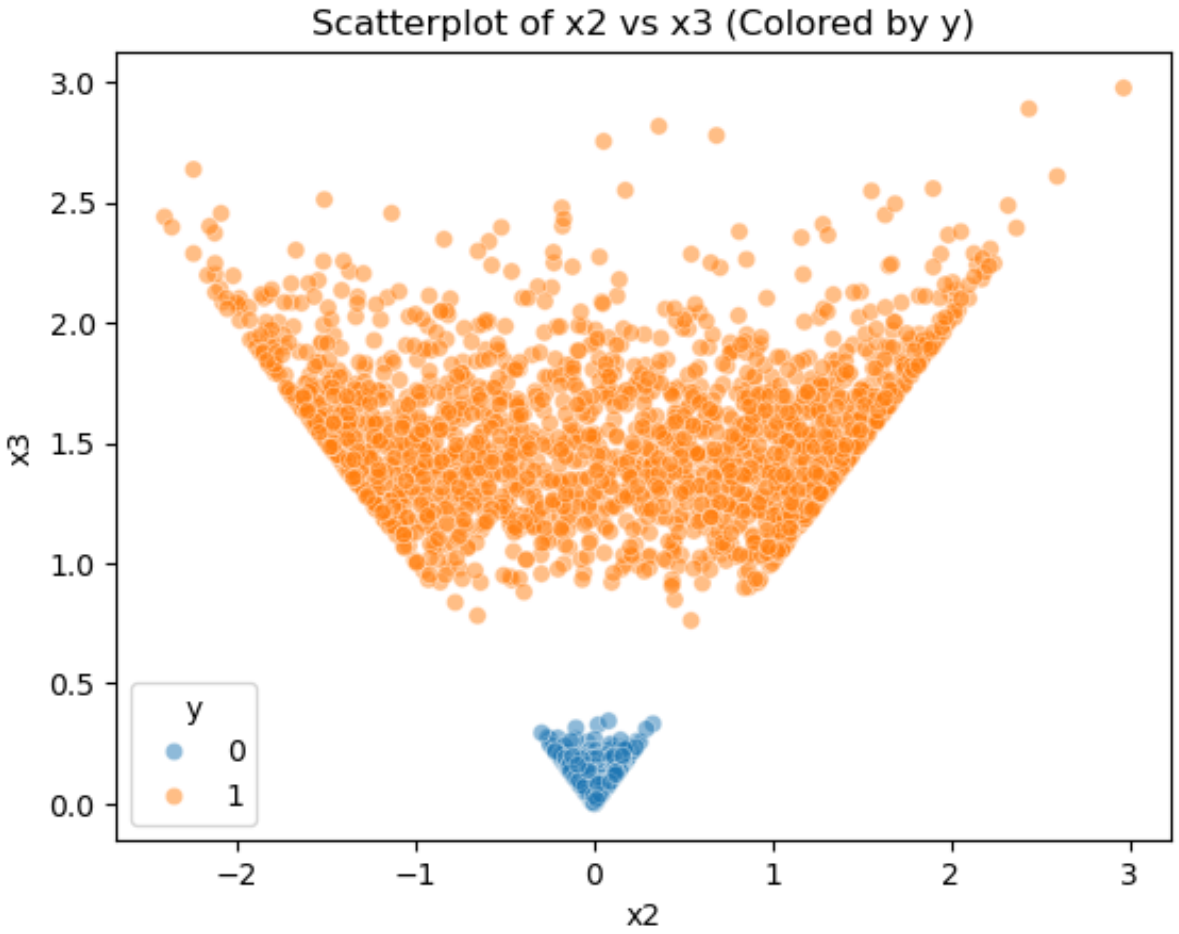
In [20]:
```
try:
    logistic_regression = LogisticRegression().fit(data[['x1', 'x2', '
except KeyError:
    print('The column `x3` does not exist!')
```

In [21]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_sc

y_pred = logistic_regression.predict(data[['x1', 'x2', 'x3']])

accuracy = accuracy_score(data['y'], y_pred)
precision = precision_score(data['y'], y_pred)
recall = recall_score(data['y'], y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
```

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
```

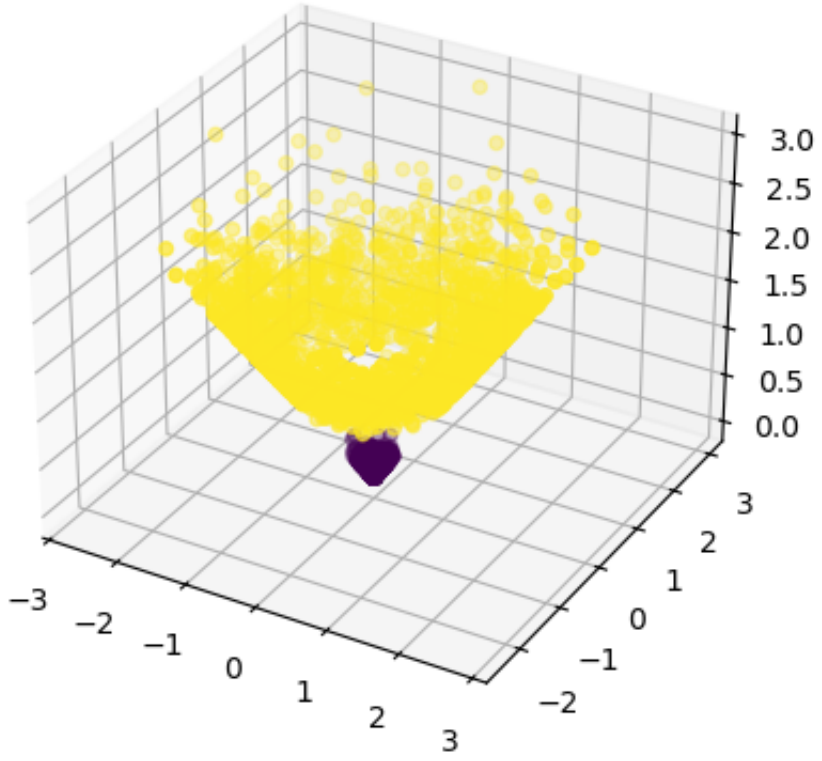
In [22]:
```
import matplotlib.pyplot as plt
import seaborn as sns


sns.scatterplot(data=data, x='x1', y='x3', hue='y', alpha=0.5)
plt.title('Scatterplot of x1 vs x3 (Colored by y)')
plt.show()


sns.scatterplot(data=data, x='x2', y='x3', hue='y', alpha=0.5)
plt.title('Scatterplot of x2 vs x3 (Colored by y)')
plt.show()
```


Scatterplot of x1 vs x3 (Colored by y)
x3
x1
y
0
1
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−2
−1
0
1
2
3



We can also visualize this in three dimensions:

In [23]:
```
try:
    _ = plt.axes(projection = '3d')
    _ = _.scatter3D(data['x1'], data['x2'], data['x3'], c=data['y'])
except KeyError:
    print('The column `x3` does not exist!')
```



Adding x3 as the distance from the origin made the classes linearly separable because points in Class 0 have small x3 and points in Class 1 have large x3.

x3 is an engineered feature. By creating it from x1 and x2 to capture the radial distance to make the classes linearly separable for logistic regression.

The SVM with the RBF kernel solved the problem automatically without creating x3, it computes distances in a higher-dimensional space using the kernel. Logistic regression with x3 is straightforward.

With hundreds or thousands of predictors, it would be difficult to manually engineer features. In that case, I would use automated feature engineering tools but also apply domain knowledge when possible, ideally combining both to create meaningful features efficiently.